SPATIAL TRANSCRIPTOMICS

A Masterclass Playbook

From Bench to Bioinformatics: A Production-Ready Guide to Next-Generation Spatial Gene Expression Analysis

Computational Biology · Molecular Biotechnology · Bioinformatics

Covers: 10x Genomics Visium · MERFISH · Xenium · scanpy · squidpy

Publication-Grade Python Pipelines · Lab Protocols · Biological Interpretation

6 Comprehensive Modules — Complete End-to-End Coverage

Table of Contents

MODULE 1 Foundations & Biological First Principles

1.1 The ELI15 Analogy: Fruit Tart, Smoothie, and the Blueprint

Understanding the three paradigms of RNA profiling requires an intuitive framing. Consider the richly structured world of a French *fruit tart*: the pastry base anchors a precise layer of custard, above which each fruit — a raspberry in the corner, a kiwi slice at centre, a blueberry cluster near the edge — occupies a defined, irreplaceable position. The tart communicates not only *what* is present but *exactly where*, and the spatial relationship between the kiwi and the raspberry is itself a piece of information.

> *The Core Analogy: BULK RNA-seq = Blending the entire tart into a smoothie and measuring average flavour. scRNA-seq = Picking off each fruit individually and identifying each one — but losing track of where it sat. Spatial Transcriptomics = Photographing the intact tart at molecular resolution: who is there AND where.*

1.1.1 Bulk RNA-seq — The Blended Smoothie

In bulk RNA-seq, an entire tissue biopsy is homogenised, RNA is extracted from the pooled lysate, and a single sequencing library is generated. The result is a **population-average transcriptome**: one number per gene representing the mean expression across potentially thousands of heterogeneous cell types. A tumour biopsy, for example, blends the signals of cancer cells, stromal fibroblasts, endothelial cells, and infiltrating T-lymphocytes into a single indistinguishable vector. Rare populations whose signal is diluted below ~1–5% of the total are effectively invisible. Critically, *all positional information is destroyed at the moment of tissue homogenisation.*

1.1.2 scRNA-seq — Picking Off Individual Fruits (But Losing the Map)

Single-cell RNA sequencing (scRNA-seq) resolves the cellular heterogeneity problem by disaggregating tissue into a suspension of individual cells, encapsulating each within a microfluidic droplet (e.g., 10x Chromium), and barcoding every transcript with a cell-specific identifier. The result is a per-cell transcriptomic atlas with single-cell resolution. However, the enzymatic or mechanical dissociation protocol required to generate a single-cell suspension **irreversibly destroys tissue architecture**. We know a FOXP3$^+$ regulatory T-cell exists, but not whether it was adjacent to a PD-L1$^+$ tumour cell or sequestered in the tumour stroma — a distinction of profound therapeutic relevance.

1.1.3 Spatial Transcriptomics — The Intact Blueprint

Spatial Transcriptomics preserves the tissue section on a solid support, captures transcripts *in situ*, and attaches a spatial barcode (an X,Y grid coordinate) to each captured mRNA. The output is a gene expression matrix in which every observation carries both a molecular identity and a physical address. This is the biological blueprint: we can now ask whether two cell types are in direct physical contact, whether a morphogen gradient decays with distance from an organising centre, or whether immune cells co-localise with or are excluded from the tumour core.

1.2 The Central Dogma in the Context of Tissue Heterogeneity

The Central Dogma of Molecular Biology describes the unidirectional flow of genetic information: **DNA** → **RNA** → **Protein**. In the context of a multicellular organism, every somatic cell carries an essentially identical genomic DNA sequence (with exceptions for somatic mutations and V(D)J recombination in lymphocytes). Yet a hepatocyte and a pancreatic beta-cell are morphologically and functionally unrecognisable as relatives. This phenotypic diversity emerges not from genomic difference but from **differential gene expression**: which subsets of genes are being actively transcribed into mRNA at any given moment in any given cell.

The mRNA (transcriptome) is therefore the informative intermediate layer — it is **dynamic, cell-type specific, and condition-responsive** in ways that the static genome is not. Sequencing mRNA rather than genomic DNA allows us to capture the *active functional state* of a cell: its current identity, stress response, differentiation trajectory, and cell-cycle phase, all at a single moment in time.

1.2.1 Why Sequence the Transient Transcriptome?

Five compelling biological and technical reasons to target mRNA:

- **Cell Identity:** Transcription factor networks define cell fate. A $CD8^+$ T-cell expresses *CD8A*, *GZMB*, and *PRF1* while a regulatory T-cell expresses *FOXP3*, *IL2RA*, and *CTLA4*. These expression signatures, invisible at the DNA level, define immunological identity.
- **Functional State:** The same neuron at rest versus during action potential propagation expresses radically different immediate-early genes (e.g., *FOS*, *ARC*, *NPAS4*). The transcriptome captures this temporal snapshot.
- **Drug Target Discovery:** Receptor and enzyme expression (rather than gene copy number) determines drug response. Differential mRNA levels of *HER2*, *EGFR*, or *PD-L1* across spatial zones directly informs therapeutic targeting.
- **Amplification Feasibility:** mRNA, once reverse-transcribed to cDNA, can be exponentially amplified by PCR, enabling detection from nanogram-scale inputs. Whole-genome DNA amplification from single cells remains technically noisier.
- **Polyadenylation Handle:** The poly(A) tail appended to mature eukaryotic mRNAs provides a universal, sequence-independent capture handle via complementary oligo-dT probes — the molecular Velcro that underpins array-based spatial capture.

1.2.2 The RNA Degradation Liability

Working with tissue-bound RNA introduces a set of degradation liabilities that have no parallel in single-cell or bulk workflows. RNA is an intrinsically labile molecule; its 2'-hydroxyl group renders the phosphodiester backbone susceptible to hydrolysis, and the ubiquitous RNase enzymes released upon cell death rapidly fragment transcripts. The relevant challenges are:

- **Ischaemic Time:** From the moment tissue perfusion ceases (surgical excision), RNases begin degrading RNA. The 'cold ischaemia time' — interval from excision to snap-freezing or fixation — should be minimised to under 30 minutes for optimal RNA Integrity Numbers (RIN $\geq$ 7).
- **Freeze-Thaw Cycles:** Each freeze-thaw cycle causes ice crystal formation that ruptures cell membranes, releasing cytoplasmic RNases. Sections should be cut fresh from the OCT block and used within the same day.
- **Lateral Diffusion:** Unlike scRNA-seq where cells are encapsulated, tissue permeabilisation releases transcripts that can diffuse laterally across the section surface before capture, blurring the spatial signal. This diffusion is governed by Brownian motion ($D \approx 10^{-7}$ cm^2/s for mRNA in aqueous medium) and must be controlled by optimised permeabilisation conditions.
- **FFPE Crosslinking:** Formalin-fixed paraffin-embedded (FFPE) tissue, the gold standard for archival pathology, creates protein-RNA crosslinks that impede reverse transcription. Specialised deparaffinisation and decrosslinking protocols (e.g., 70°C citrate buffer incubation) are required.

- **Section Thickness:** Optimal cryosection thickness for Visium is 10 µm. Sections that are too thick increase the probability of multi-layer transcript signal; too thin reduces total RNA yield below detection thresholds.

MODULE 2

Comparative Analysis, Benefits & Real-World Applications

2.1 The Technology Landscape

The following table provides a rigorous, multi-dimensional comparison of the four principal transcriptomic profiling paradigms relevant to modern biomedical research. Values reflect current state-of-the-art commercial platforms (2023–2025).

Attribute	Bulk RNA-seq	scRNA-seq (10x Chromium)	Seq-based ST (10x Visium)	Imaging-based ST (MERFISH / Xenium)
Spatial Resolution	None (tissue homogenate)	None (dissociated cells)	~55 µm spot (multi-cellular)	Sub-cellular (< 1 µm)
Transcriptomic Depth	Whole transcriptome (~20,000 genes)	Whole transcriptome (~20,000 genes)	Whole transcriptome (~33,000 genes)	Targeted panel (100–5,000 genes)
Cells / Spots per Run	1 bulk measurement	500–20,000 cells	~3,000–5,000 spots	100,000–1M+ cells
Capture Efficiency	High (µg RNA input)	~10–20% of mRNA	~10–20% of mRNA	~20–80% (FISH-based)
Single-Cell Resolution	No	Yes	No (spot averages)	Yes
Tissue Architecture Preserved	No	No	Yes	Yes
Multiplexing Capacity	High	Moderate (hashtag)	Moderate	Low–moderate
Data Analysis Complexity	Low	Moderate	Moderate-High	High
Reagent Cost (per sample)	$150–400	$1,000–2,500	$600–1,500	$2,000–6,000
Instrument Required	Sequencer only	10x Chromium + Seq	10x Visium slide + Seq	Dedicated imaging sys.
Key Limitation	No cell-type resolution; no spatial info	No spatial context; dissociation bias	55 µm spot averages multiple cell types	Limited panel size; high cost per gene
Optimal Use Case	Bulk DE, gene signatures, cohorts	Cell-type atlas, trajectory analysis	Spatial expression, cell neighbourhood	Sub-cellular localisation, clinical diagnostics

Table 1: Multi-dimensional comparison of RNA profiling technologies. Costs are approximate and platform-dependent.

2.2 Core Benefits of Spatial Transcriptomics

2.2.1 Resolving the Tumour Microenvironment (TME)

The TME is a complex ecosystem comprising malignant cells, cancer-associated fibroblasts (CAFs), tumour-associated macrophages (TAMs), endothelial cells, and infiltrating lymphocytes. Bulk RNA-seq averages this ecosystem; scRNA-seq identifies each player but discards spatial context. ST reveals the TME's **functional**

geography: which immune populations infiltrate the tumour core versus are excluded to the invasive margin; whether PD-L1 expression on tumour cells spatially correlates with $CD8^+$ T-cell proximity; and how CAF subtypes partition between perivascular and desmoplastic niches. These spatial patterns are mechanistically meaningful — an 'excluded' immune phenotype predicts checkpoint immunotherapy resistance regardless of total immune infiltrate.

2.2.2 Identifying Spatial Niches and Cellular Neighbourhoods

A spatial niche is a reproducible, co-occurring cluster of cell types that jointly constitute a functional unit — analogous to a town with its shops, houses, and roads always appearing together. ST enables **neighbourhood enrichment analysis** to quantify which cell types are physically co-localised beyond random chance. In the liver, for example, pericentral hepatocytes constitute a metabolic niche characterised by WNT target gene expression, while periportal hepatocytes express gluconeogenic enzymes — a zonation pattern invisible to non-spatial methods.

2.2.3 Mapping Morphogen Gradients and Signalling Axes

Developmental patterning relies on graded extracellular signals (morphogens such as WNT, BMP, SHH, and RA) that establish positional identity across tissue fields. ST can directly visualise these gradients as continuous gene expression profiles across space, bridging classical developmental biology with single-molecule precision. Similarly, in mature tissues, paracrine signalling gradients — such as oxygen tension gradients driving HIF-1α target gene activation in tumour hypoxic zones — can be mapped and correlated with pathological phenotypes.

2.3 Clinical and Research Applications

2.3.1 Oncology: Mapping Immune Checkpoint Spatial Distribution

Immune checkpoint blockade (ICB) therapy targeting PD-1/PD-L1 and CTLA-4 axes has transformed cancer treatment, yet response rates remain below 30% in most solid tumour types. ST addresses a fundamental gap in understanding resistance: the spatial distribution of checkpoint molecules within the TME. Key applications include:

- **PD-L1 Spatial Heterogeneity:** PD-L1 (CD274) expression varies within a single tumour section — high at the invasive front, low in the hypoxic core. Immunohistochemistry (IHC) scores this heterogeneity as a single biopsy average; ST resolves it spatially, revealing that $CD8^+$ T-cells preferentially localise to PD-$L1^{high}$ zones, validating the mechanistic rationale for anti-PD-1 therapy in those regions.
- **Myeloid Exclusion Zones:** In pancreatic ductal adenocarcinoma (PDAC), ST reveals a dense, immunosuppressive myeloid barrier surrounding tumour nests, explaining why T-cells fail to penetrate despite systemic priming. This spatial information directly motivates combination strategies targeting $CSF1R^+$ macrophage depletion.
- **Spatial Clonal Dynamics:** Integration of ST with somatic mutation data allows mapping of mutant clones to specific spatial zones, revealing selection gradients across the tumour microenvironment.

2.3.2 Neuroscience: Transcriptomic Layers of the Cerebral Cortex

The mammalian neocortex is organised into six functionally distinct cellular layers (L1–L6) defined by characteristic cell types, connectivity patterns, and gene expression profiles. Classic histology identifies these layers morphologically; ST provides the molecular underpinning:

- **Layer-Specific Markers:** RORB (Layer 4 excitatory neurons), THEMIS (Layer 5/6), CUX1/CUX2 (Layers 2/3), and TBR1 (deep layers) can be mapped with ST, validating and extending cytoarchitectural atlases such as the Allen Brain Atlas.

- **Disease Perturbation Mapping:** In Alzheimer's disease, ST reveals that amyloid plaques are surrounded by a reactive astrocyte halo expressing a specific gene module (*GFAP*, *LCN2*, *SERPINA3*) that differs from homeostatic astrocytes elsewhere in the section — a spatial cellular response invisible to bulk analysis.
- **Circuit-Level Spatial Connectomics:** Emerging multi-modal approaches combine ST with retrograde tracing to map not only what a neuron expresses but which long-range circuits it participates in.

2.3.3 Developmental Biology: Morphogen Gradients During Organogenesis

Embryogenesis is fundamentally a spatial problem: a single fertilised egg must generate hundreds of cell types in precise three-dimensional arrangements. ST provides an unprecedented window into this process:

- **Gastrulation Axes:** In the mouse embryo at E7.5, ST reveals the opposing anterior-posterior BMP and WNT gradients that define body axis specification, resolving a long-standing question about the spatial extent of organiser activity.
- **Organ Patterning:** During kidney nephrogenesis, the branching ureteric bud expresses a spatially restricted WNT9b gradient that induces mesenchyme-to-epithelium transition (MET) in adjacent metanephric mesenchyme. ST directly visualises this inductive interaction at the molecular level.
- **Temporal-Spatial Atlases:** Landmark projects such as the Spatiotemporal Transcriptomic Atlas of Mouse Organogenesis (STAMO) use ST across developmental time points to construct four-dimensional gene expression maps, serving as reference frameworks for understanding congenital disease mechanisms.

MODULE 3 Bench Science & Lab Chemistry (Step-by-Step)

This module provides a chronological walk-through of the physical laboratory workflow for sequencing-based Spatial Transcriptomics using the 10x Genomics Visium platform. Each step is annotated with the critical quality control metrics and failure modes that a practising scientist must monitor.

Step 1 Tissue Cryosectioning & Fixation

- Embed fresh-frozen tissue in Optimal Cutting Temperature (OCT) compound within a cryomold and snap-freeze on dry ice or liquid nitrogen. Store at –80°C until sectioning.
- Set cryostat chamber temperature to –20°C (±2°C for most soft tissues; –15°C for brain to prevent tearing; –22°C for adipose tissue). Object temperature should match or be 2–3°C colder than the chamber.
- **Section thickness: 10 µm** (Visium specification). Use anti-roll plate or brush to flatten sections. Mount directly onto the pre-chilled Visium capture area.
- Fix sections immediately by submerging slides in pre-chilled methanol (–20°C) for exactly **30 minutes**. Methanol fixation preserves RNA integrity while removing lipids that impede permeabilisation. Avoid formalin at this stage.
- **QC Metric:** RNA Integrity Number (RIN) ≥ 7 from a parallel section. DV200 score (% fragments > 200 nt) ≥ 50% for FFPE sections.

Step 2 Histology Staining & High-Resolution Imaging

- After methanol fixation, proceed to Haematoxylin and Eosin (H&E;) staining or immunofluorescence (IF). H&E; is standard for tissue morphology annotation; IF provides cell-type-specific protein markers for co-registration.
- H&E; Protocol (brief): Rehydrate → Haematoxylin (nuclei stain blue, 7 min) → Eosin (cytoplasm/ECM stain pink, 1 min) → Dehydrate → Mount coverslip.
- Acquire brightfield or fluorescence images at **20× magnification minimum** using a slide scanner (e.g., Zeiss Axioscan, Leica Aperio). Target pixel resolution: 0.25–0.5 µm/pixel. Images are registered against the Visium capture area fiducial frame for spot alignment.
- **Critical:** Image acquisition must occur *before* permeabilisation. Permeabilisation disrupts tissue morphology. The high-resolution image serves as the permanent spatial anchor for all downstream transcript mapping.
- **Output:** A TIFF image file (.tif) that is input into the Space Ranger alignment pipeline.

Step 3 Permeabilisation Optimisation

- Permeabilisation is the most technically variable and critical step. The goal is to create aqueous channels through the cell membrane and ECM, allowing cytoplasmic mRNA to diffuse downward onto the capture array while minimising lateral diffusion.
- The 10x Visium Tissue Optimisation slide provides a matrix of enzyme concentrations × time (1–60 min permeabilisation enzyme) to empirically determine the optimal condition for each tissue type.
- **Enzymatic mechanism:** Protease K (or collagenase for dense ECM tissues) cleaves peptide bonds in membrane proteins, creating pores of sufficient diameter (~10–50 nm) for mRNA transit. Kinetics follow a Michaelis-Menten model; substrate concentration (tissue protein density) determines Vmax.

- **Lateral diffusion physics:** Released mRNA diffuses with Brownian motion: mean-squared displacement $\sigma^2 = 2Dt$, where $D \approx 10^{-8}$ cm^2/s for mRNA in cytoplasm-like medium. At 15 min permeabilisation, σ ≈ 5–15 µm — acceptable for 55 µm spots but catastrophic if permeabilisation is over-extended.
- **Tissue-specific guidelines:** Brain: 12 min **|** Breast tumour: 24 min **|** Liver: 18 min **|** Skin: 30 min **|** Kidney: 18 min. Always validate empirically.
- **QC Metric:** Fluorescent cDNA signal on optimisation slide should be intense, circular (not diffuse halos), and confined within tissue boundaries.

Step 4 mRNA Capture — Physics of Transcript Falling onto the Array

- The Visium capture slide contains **~5,000 capture spots**, each 55 µm in diameter and containing ~250,000 barcoded capture probes. Spots are arranged in a hexagonal lattice with 100 µm centre-to-centre spacing.
- Capture probes contain a $poly(T)_{30}$ tail that hybridises to the poly(A) tail of mature eukaryotic mRNA by complementary base-pairing (Watson-Crick hybridisation: ΔG ≈ –1.5 kcal/mol per A:T pair × 30 pairs ≈ –45 kcal/mol at 37°C).
- Following permeabilisation, a hybridisation buffer is applied. Gravity, convection, and diffusion drive transcripts down through the 10 µm section onto the array surface. Capture incubation: **37°C for 30 minutes**.
- Captured transcripts are physically anchored to their originating spot's X,Y coordinate, permanently linking the transcript's molecular identity to its tissue address.
- **Capture efficiency:** Approximately 10–20% of $poly(A)^+$ transcripts are captured per spot. This is comparable to scRNA-seq droplet platforms and reflects both hybridisation kinetics and transcript accessibility.

Step 5 Molecular Barcoding — Anatomy of the Capture Probe

- Each capture probe is a precisely engineered oligonucleotide with four functional domains in 5'→3' order:
- **Domain 1 — Partial Read 1 Sequence (≈28 nt):** Illumina sequencing primer binding site, enabling the sequencer to initiate reading from the spatial barcode end. Constant across all probes on the array.
- **Domain 2 — Spatial Barcode (16 nt):** The X,Y 'zip code' — a unique 16-nucleotide sequence identifying the specific spot on the array. All probes within one spot share an identical spatial barcode. The codebook of spatial barcode → X,Y coordinates is provided by 10x Genomics and used by Space Ranger for spot demultiplexing.
- **Domain 3 — Unique Molecular Identifier (UMI, 12 nt):** A random 12-nucleotide sequence that is unique to each individual probe molecule. PCR amplification creates many copies of each captured transcript, but they all share the same UMI. Collapsing reads by UMI eliminates PCR duplicates, providing a count of original transcript molecules rather than amplified copies. Theoretical UMI complexity: $4^{12} = 16,777,216$ unique identifiers.
- **Domain 4 — Poly(T) Capture Tail (T_{30}VN):** 30 thymidines followed by a degenerate VN anchor (V = A, C, or G; N = any base) that prevents internal priming and anchors the probe at the beginning of the poly(A) tail.
- After hybridisation and capture, **reverse transcriptase** extends from the poly(T) tail through the mRNA, incorporating the spatial barcode and UMI into the first-strand cDNA. The mRNA is then degraded (RNase H), leaving the barcoded cDNA permanently attached to its spot.

Step 6 Library Preparation & Next-Generation Sequencing (NGS)

- **On-slide cDNA synthesis:** Second-strand synthesis and denaturation release the double-stranded cDNA from the slide surface. The cDNA is collected from all spots collectively into a single tube.

- **cDNA amplification:** PCR amplification (12–16 cycles) generates sufficient material for Illumina library preparation. Cycle number is determined by the total cDNA yield measured by Bioanalyzer (target: >1 ng/µL, average fragment size ~500–600 bp).
- **Library fragmentation and adapter ligation:** Enzymatic fragmentation + end-repair + A-tailing + P5/P7 Illumina adapter ligation + indexing PCR. Final library fragment size distribution: 300–500 bp.
- **Paired-end sequencing:** Libraries are sequenced on NovaSeq 6000/X Plus using a 28+90 bp paired-end configuration: Read 1 (28 bp) sequences the Spatial Barcode (16 bp) + UMI (12 bp); Read 2 (90 bp) sequences the cDNA insert for gene identification by alignment to the genome.
- **Recommended sequencing depth:** ~25,000 reads per spot × 4,000 spots = ~100 million reads per sample for standard human tissue. Brain tissue, with higher transcriptional complexity, benefits from 50,000 reads/spot.
- **Bioinformatics preprocessing:** Space Ranger (10x Genomics) demultiplexes spatial barcodes, aligns Read 2 to the reference genome (STARsolo alignment), counts UMIs per gene per spot, and registers spots to the tissue image, producing a filtered feature-barcode matrix and tissue position list as outputs.

MODULE 4

The Step-by-Step Analysis Pipeline (Python Code)

The following is a complete, production-ready, end-to-end Python pipeline for analysing a 10x Genomics Visium Spatial Transcriptomics dataset. The pipeline uses the AnnData data structure as its central data container and leverages the scanpy and squidpy ecosystems. Each section is annotated with the biological rationale for every operation.

4.1 Environment Setup & Dependencies

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# SPATIAL TRANSCRIPTOMICS ANALYSIS PIPELINE
# Platform: 10x Genomics Visium
# Dependencies: scanpy>=1.9, squidpy>=1.3, matplotlib, seaborn, pandas, numpy
# Python: 3.9+
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

# ■■ 1. IMPORTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import os
import warnings
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns

import scanpy as sc
import squidpy as sq

from pathlib import Path
from scipy.sparse import issparse

warnings.filterwarnings("ignore")
matplotlib.rcParams["figure.dpi"] = 150   # screen preview DPI
sc.settings.verbosity = 2                 # 0=errors, 1=warnings, 2=info, 3=debug
sc.logging.print_header()                 # log software versions for reproducibility
sc.settings.set_figure_params(
    dpi=300,
    facecolor="white",
    frameon=False,
)
```

4.2 Data Loading

```
# ■■ 2. DATA LOADING ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Option A: Load from a local Space Ranger output directory
#   space_ranger_output/
#     ■■■ filtered_feature_bc_matrix/
#     ■     ■■■ barcodes.tsv.gz
```

```
#     ■     ■■■ features.tsv.gz
#     ■     ■■■ matrix.mtx.gz
#     ■■■ spatial/
#     ■     ■■■ tissue_positions_list.csv
#     ■     ■■■ scalefactors_json.json
#     ■     ■■■ tissue_hires_image.png
#     ■     ■■■ tissue_lowres_image.png

DATA_DIR = Path("./visium_output")       # replace with your Space Ranger output path
RESULTS_DIR = Path("./results")
RESULTS_DIR.mkdir(exist_ok=True)

# sc.read_visium reads the filtered_feature_bc_matrix + spatial folder
# and constructs a complete AnnData object with spatial coordinates embedded
adata = sc.read_visium(
    path=DATA_DIR,
    count_file="filtered_feature_bc_matrix.h5",
    load_images=True,
    source_image_path=DATA_DIR / "spatial" / "tissue_hires_image.png",
)

# Option B: Load the public Human Lymph Node Visium dataset from scanpy
# (Uncomment these two lines if you do not have a local dataset)
# sc.datasets.visium_sge(sample_id="V1_Human_Lymph_Node", save_path=DATA_DIR)
# adata = sc.read_visium(DATA_DIR / "V1_Human_Lymph_Node")

# Ensure gene names (var_names) are unique and human-readable
adata.var_names_make_unique()

# Convert spot coordinates to float for downstream spatial computations
adata.obsm["spatial"] = adata.obsm["spatial"].astype(float)

print(f"AnnData object loaded: {adata.n_obs} spots x {adata.n_vars} genes")
print(adata)
```

4.3 Quality Control

```
# ■■ 3. QUALITY CONTROL (QC) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 3a. Annotate mitochondrial genes (high mt% = damaged/apoptotic cells)
#     Human mitochondrial genes are prefixed "MT-"; mouse genes "mt-"
adata.var["mt"] = adata.var_names.str.startswith("MT-")

# 3b. Annotate ribosomal genes (RPS = small subunit, RPL = large subunit)
adata.var["ribo"] = adata.var_names.str.startswith(("RPS", "RPL"))

# 3c. Annotate haemoglobin genes (proxy for red blood cell contamination)
adata.var["hb"] = adata.var_names.str.contains("^HB[^(P)]", regex=True)

# 3d. Compute per-spot QC metrics using scanpy's built-in function
#     This populates adata.obs with:
#     - total_counts        : total UMI counts per spot
#     - n_genes_by_counts   : number of detected genes per spot
#     - pct_counts_mt       : % of UMIs from mitochondrial genes
#     - pct_counts_ribo     : % of UMIs from ribosomal genes
sc.pp.calculate_qc_metrics(
    adata,
```

```
    qc_vars=["mt", "ribo", "hb"],
    percent_top=None,
    log1p=False,
    inplace=True,
)

# 3e. Inspect QC metric distributions
print("\n■■ QC Summary Statistics ■■")
print(adata.obs[["total_counts","n_genes_by_counts","pct_counts_mt"]].describe())
```

4.4 Filtering

```
# ■■ 4. FILTERING OUTLIER SPOTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Strategy: Remove spots below minimum thresholds (low-quality / empty spots)
# and above maximum thresholds (doublets / contaminated spots).
# Thresholds below are starting points; adjust empirically per dataset.

# 4a. Define filtering thresholds
MIN_COUNTS   = 500        # minimum total UMI counts per spot
MIN_GENES    = 200        # minimum detected genes per spot
MAX_PCT_MT   = 20         # maximum mitochondrial read fraction (%)

# 4b. Apply cell-level (spot-level) filters
print(f"Spots before filtering: {adata.n_obs}")
sc.pp.filter_cells(adata, min_counts=MIN_COUNTS)
sc.pp.filter_cells(adata, min_genes=MIN_GENES)
adata = adata[adata.obs["pct_counts_mt"] < MAX_PCT_MT].copy()
print(f"Spots after filtering:  {adata.n_obs}")

# 4c. Apply gene-level filters
# Remove genes detected in fewer than 10 spots (low-information genes)
sc.pp.filter_genes(adata, min_cells=10)
print(f"Genes after filtering:  {adata.n_vars}")
```

4.5 Normalisation & Log Transformation

```
# ■■ 5. NORMALISATION & LOG TRANSFORMATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 5a. Store the raw count matrix before any normalisation
#     adata.layers["counts"] preserves the integer UMI matrix for downstream
#     differential expression analysis (DESeq2 / edgeR require raw counts)
adata.layers["counts"] = adata.X.copy()

# 5b. Library-size normalisation (total-count normalisation)
#     Scales each spot's counts so that the total UMI count equals 10,000.
#     This removes the technical variation in sequencing depth between spots,
#     converting raw counts into Counts Per 10,000 (CP10K / CPM-equivalent).
#     Normalisation factor for spot i: scale_i = 10,000 / sum(counts_i)
sc.pp.normalize_total(
    adata,
    target_sum=1e4,         # normalise to 10,000 counts per spot
    inplace=True,
)

# 5c. Log1p transformation: X_new = log(X + 1)
#     Compresses the dynamic range of expression values (which span several
#     orders of magnitude), making the data more normally distributed and
```

```
#     stabilising variance. Adding 1 before logging avoids log(0) = -inf.
sc.pp.log1p(adata)

# 5d. Store normalised log-transformed matrix in a dedicated layer
adata.layers["lognorm"] = adata.X.copy()

print("Normalisation complete. adata.X now holds log1p(CP10K) values.")
```

4.6 Highly Variable Gene (HVG) Selection

```
# ■■ 6. HIGHLY VARIABLE GENE (HVG) SELECTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Identify genes whose expression varies substantially across spots —
# these genes carry the biological signal that distinguishes cell types.
#
# Method: Seurat v3 / scanpy implementation
#   1. Bin genes by mean expression into n_top_genes bins
#   2. Within each bin, compute the variance of the normalised expression
#   3. Standardise variance relative to the expected variance for that
#      expression level (variance-to-mean ratio, or Fano factor)
#   4. Select the top n_top_genes with highest standardised variance
sc.pp.highly_variable_genes(
    adata,
    flavor="seurat_v3",          # use Seurat v3 variance-stabilisation method
    n_top_genes=3000,             # retain top 3,000 HVGs for PCA
    layer="counts",               # HVG selection on raw counts (Seurat v3 requirement)
    subset=False,                 # annotate but don't subset yet
)

print(f"HVGs identified: {adata.var['highly_variable'].sum()} / {adata.n_vars}")

# Optional: inspect HVG distribution
# sc.pl.highly_variable_genes(adata)
```

4.7 Dimensionality Reduction: PCA → Graph → UMAP

```
# ■■ 7. DIMENSIONALITY REDUCTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 7a. Scale gene expression to zero mean and unit variance
#     This prevents high-magnitude genes from dominating PCA directions.
#     max_value=10 clips extreme outliers (prevents a single cell from
#     pulling a PC direction).
sc.pp.scale(adata, max_value=10)

# 7b. Principal Component Analysis (PCA)
#     Reduces the ~3,000 HVG-dimensional space to n_comps principal
#     components. Uses SVD decomposition on the scaled expression matrix.
#     Result stored in adata.obsm['X_pca'] (n_spots × n_comps)
#                    and adata.varm['PCs']  (n_genes × n_comps)
sc.tl.pca(
    adata,
    n_comps=50,              # compute top 50 PCs
    use_highly_variable=True,  # use only HVGs for PCA
    svd_solver="arpack",     # ARPACK for sparse matrices (memory efficient)
    random_state=42,
)

# 7c. Elbow plot to determine the number of informative PCs
```

```
sc.pl.pca_variance_ratio(adata, n_pcs=50, log=True,
                          save="_elbow_plot.png")

# 7d. Nearest-Neighbour Graph construction
#     Constructs a k-NN graph in the PCA embedding space where each spot
#     is connected to its n_neighbors=15 most similar spots.
#     Distances computed using Euclidean metric in PC space.
#     Result stored in adata.obsp['connectivities'] and adata.obsp['distances']
sc.pp.neighbors(
    adata,
    n_neighbors=15,         # k in k-NN graph
    n_pcs=30,               # use top 30 PCs (informed by elbow plot)
    random_state=42,
)

# 7e. UMAP (Uniform Manifold Approximation and Projection)
#     2D non-linear embedding for visualisation, computed from the k-NN graph.
#     UMAP preserves both local (cluster) and global (inter-cluster) structure
#     better than t-SNE. Result in adata.obsm['X_umap'].
sc.tl.umap(
    adata,
    min_dist=0.3,     # minimum distance between points in UMAP space
    spread=1.0,       # effective scale of embedded points
    random_state=42,
)
```

4.8 Leiden Clustering

```
# ══ 8. LEIDEN CLUSTERING ══════════════════════════════════════════════════════════════
# The Leiden algorithm is a community detection method that optimises
# modularity on the k-NN graph. It is a refinement of the Louvain algorithm
# with guaranteed well-connected communities.
#
# Resolution parameter controls granularity:
#   - Higher resolution → more, smaller clusters
#   - Lower resolution → fewer, larger clusters
# Typical range: 0.3 (coarse) to 1.5 (fine) for Visium data

sc.tl.leiden(
    adata,
    resolution=0.6,       # adjust based on biological question
    random_state=42,
    key_added="leiden",   # stored in adata.obs['leiden']
)

n_clusters = adata.obs["leiden"].nunique()
print(f"Leiden clustering at resolution=0.6 identified {n_clusters} clusters.")
print("Cluster cell counts:")
print(adata.obs["leiden"].value_counts().sort_index())
```

4.9 Spatial Analysis with Squidpy

```
# ══ 9. SPATIAL GRAPH & NEIGHBOURHOOD ENRICHMENT (squidpy) ══════════════════
# 9a. Build a spatial neighbourhood graph based on physical coordinates
#     (distinct from the transcriptomic k-NN graph above).
#     coord_type='visium' uses the hexagonal Visium spot layout.
```

```
sq.gr.spatial_neighbors(
    adata,
    coord_type="visium",
    n_neighs=6,             # 6 direct neighbours in hexagonal lattice
    key_added="spatial_connectivities",
)

# 9b. Neighbourhood enrichment analysis
#     Computes how often each pair of Leiden clusters appears as
#     physical neighbours vs. random expectation.
#     Uses permutation testing (n_perms=1000) to compute z-scores.
#     Result stored in adata.uns['leiden_nhood_enrichment']
sq.gr.nhood_enrichment(
    adata,
    cluster_key="leiden",
    n_perms=1000,
    seed=42,
)

# 9c. Ripley's L statistic (spatial autocorrelation of cluster patterns)
sq.gr.ripley(
    adata,
    cluster_key="leiden",
    mode="L",
    max_dist=500,           # maximum distance in micrometres
)

print("Spatial analyses complete.")
```

4.10 Marker Gene Identification

```
# ── 10. DIFFERENTIAL EXPRESSION — CLUSTER MARKER GENES ───────────────────
# Use the raw count layer for DE analysis (scanpy's rank_genes_groups
# uses Wilcoxon rank-sum test by default — non-parametric, robust to
# non-normality of single-spot count distributions).
# Restores the raw count matrix to adata.X for DE:
adata.X = adata.layers["counts"].copy()

sc.tl.rank_genes_groups(
    adata,
    groupby="leiden",         # compare each cluster vs. all others
    method="wilcoxon",        # Wilcoxon rank-sum test
    use_raw=False,
    layer="counts",           # use raw counts for the test
    n_genes=50,               # report top 50 markers per cluster
    pts=True,                 # compute % of spots expressing each gene
    key_added="rank_genes_groups",
)

# Restore log-normalised matrix for visualisation
adata.X = adata.layers["lognorm"].copy()

# Extract results as a DataFrame
marker_df = sc.get.rank_genes_groups_df(
    adata,
    group=None,             # None = all groups
```

```
    key="rank_genes_groups",
    pval_cutoff=0.05,
    log2fc_min=0.5,
)
print(f"DE results shape: {marker_df.shape}")
print(marker_df.head(10))
marker_df.to_csv(RESULTS_DIR / "cluster_marker_genes.csv", index=False)
```

4.11 Publication-Ready Visualisation

```
# ■■ 11. PUBLICATION-READY MULTI-PANEL FIGURE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Generate a 6-panel figure suitable for a main figure in a publication.
# All settings use explicit DPI, clean palettes, and proper labelling.

PALETTE = "tab20"          # 20-colour categorical palette for clusters
SPATIAL_CMAP = "magma"     # sequential colourmap for continuous values
FIG_DPI = 300              # publication-grade DPI
ALPHA = 0.85               # spot transparency

fig = plt.figure(figsize=(18, 14), dpi=FIG_DPI, facecolor="white")
fig.suptitle(
    "Spatial Transcriptomics Analysis — 10x Visium",
    fontsize=16, fontweight="bold", y=0.98
)
gs = gridspec.GridSpec(
    2, 3,
    figure=fig,
    hspace=0.35,
    wspace=0.30,
    left=0.05, right=0.97,
    top=0.93, bottom=0.05,
)

# ■■ Panel A: Spatial map of Leiden clusters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax_a = fig.add_subplot(gs[0, 0])
sc.pl.spatial(
    adata,
    color="leiden",
    palette=PALETTE,
    alpha=ALPHA,
    size=1.5,
    ax=ax_a,
    show=False,
    title="A  Leiden Clusters (Spatial)",
    legend_loc="right margin",
    frameon=False,
)
ax_a.set_xlabel("X coordinate (μm)", fontsize=9)
ax_a.set_ylabel("Y coordinate (μm)", fontsize=9)

# ■■ Panel B: UMAP coloured by Leiden clusters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax_b = fig.add_subplot(gs[0, 1])
sc.pl.umap(
    adata,
    color="leiden",
    palette=PALETTE,
```

```
        alpha=ALPHA,
        ax=ax_b,
        show=False,
        title="B  UMAP — Leiden Clusters",
        legend_loc="right margin",
        frameon=False,
        size=8,
    )
    ax_b.set_xlabel("UMAP 1", fontsize=9)
    ax_b.set_ylabel("UMAP 2", fontsize=9)

    # ■■ Panel C: Spatial map of total UMI counts (QC proxy) ■■■■■■■■■■■■■■■■■■■■■■■■
    ax_c = fig.add_subplot(gs[0, 2])
    sc.pl.spatial(
        adata,
        color="total_counts",
        cmap=SPATIAL_CMAP,
        alpha=ALPHA,
        size=1.5,
        ax=ax_c,
        show=False,
        title="C  Total UMI Counts per Spot",
        frameon=False,
    )
    ax_c.set_xlabel("X coordinate (µm)", fontsize=9)
    ax_c.set_ylabel("", fontsize=9)

    # ■■ Panel D: Spatial expression of a canonical marker gene ■■■■■■■■■■■■■■■■■■■
    # Replace "MS4A1" (B-cell marker) with a gene relevant to your tissue
    EXAMPLE_GENE = "MS4A1"
    if EXAMPLE_GENE in adata.var_names:
        ax_d = fig.add_subplot(gs[1, 0])
        sc.pl.spatial(
            adata,
            color=EXAMPLE_GENE,
            cmap="viridis",
            alpha=ALPHA,
            size=1.5,
            ax=ax_d,
            show=False,
            title=f"D  Spatial Expression: {EXAMPLE_GENE}",
            frameon=False,
            vmin=0,
            vmax=adata[:, EXAMPLE_GENE].X.toarray().max() if issparse(adata.X) else
                 adata[:, EXAMPLE_GENE].X.max(),
        )
        ax_d.set_xlabel("X coordinate (µm)", fontsize=9)
        ax_d.set_ylabel("Y coordinate (µm)", fontsize=9)

    # ■■ Panel E: Neighbourhood enrichment heatmap ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    ax_e = fig.add_subplot(gs[1, 1])
    sq.pl.nhood_enrichment(
        adata,
        cluster_key="leiden",
        ax=ax_e,
        title="E  Neighbourhood Enrichment Z-score",
        figsize=None,
```

```
    cmap="RdBu_r",
    vmin=-50, vmax=50,
)

# ■■ Panel F: Dot plot — top marker genes per cluster ■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Extract top 2 marker genes per cluster
top_markers = []
for grp in sorted(adata.obs["leiden"].unique(), key=int):
    df_grp = marker_df[marker_df["group"] == grp].head(2)
    top_markers.extend(df_grp["names"].tolist())
top_markers = list(dict.fromkeys(top_markers))[:20]  # unique, max 20

ax_f = fig.add_subplot(gs[1, 2])
sc.pl.dotplot(
    adata,
    var_names=top_markers,
    groupby="leiden",
    ax=ax_f,
    show=False,
    title="F  Top Marker Genes per Cluster",
    standard_scale="var",
    cmap="RdPu",
    dot_max=0.5,
    colorbar_title="Mean Expression
(scaled)",
)

plt.tight_layout()
out_fig = RESULTS_DIR / "ST_analysis_figure.pdf"
fig.savefig(out_fig, dpi=FIG_DPI, bbox_inches="tight", facecolor="white")
out_png = RESULTS_DIR / "ST_analysis_figure.png"
fig.savefig(out_png, dpi=FIG_DPI, bbox_inches="tight", facecolor="white")
print(f"Figure saved to: {out_fig}")
plt.close()

# ■■ Save the final AnnData object ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
adata.write_h5ad(RESULTS_DIR / "visium_analysed.h5ad")
print("Analysis complete. AnnData saved.")
```

MODULE 5

Deep-Dive Line-by-Line Code Matrix

The following matrix provides an exhaustive breakdown of every operational step in the Module 4 pipeline. For each line, it details the input state, output state (including exactly where in the AnnData object data is stored), and the scientific/mathematical rationale for the operation.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`import scanpy as sc`	Python interpreter. No AnnData exists yet.	sc namespace loaded into memory. Provides sc.pp, sc.tl, sc.pl sub-namespaces.	scanpy is the foundational AnnData-centric toolkit for single-cell and spatial analysis. Loads ~150 functions across preprocessing, tools, and plotting.
`import squidpy as sq`	Python interpreter.	sq namespace loaded. Provides sq.gr (graph), sq.pl (plot), sq.im (image) sub-namespaces.	squidpy extends scanpy with spatial-specific analyses: neighbourhood enrichment, spatial autocorrelation (Moran's I), Ripley statistics, and image feature extraction.
`sc.settings.set_figure_params(dpi=300,...)`	Default matplotlib rcParams (dpi=72).	Updates matplotlib.rcParams: figure.dpi=300, facecolor='white', frameon=False.	DPI=300 ensures figures meet Nature/Cell figure submission requirements (minimum 300 dpi). frameon=False removes decorative box borders for clean academic aesthetics.
`adata = sc.read_visium(path=DATA_DIR, ...)`	Space Ranger output directory with filtered_feature_bc_matrix.h5 and spatial/ folder.	adata.X: sparse count matrix (n_spots × n_genes). adata.obs: spot metadata. adata.var: gene metadata. adata.obsm['spatial']: (n_spots × 2) pixel coordinate array. adata.uns['spatial']: dict containing tissue images and scale factors.	sc.read_visium parses the HDF5 count matrix, the tissue position CSV (mapping barcode → pixel X,Y), and embeds the high-resolution tissue image. This creates a unified object where every spot's gene expression is linked to its physical tissue coordinate.
`adata.var_names_make_unique()`	adata.var.index may contain duplicate Ensembl IDs or gene symbols.	adata.var.index: all gene names are now unique (duplicates suffixed with -1, -2, etc.).	Duplicate gene names cause indexing ambiguities in all downstream operations. This is a mandatory preprocessing step, particularly after gene ID to symbol mapping.
`adata.obsm['spatial'] = adata.obsm['spatial'].astype(float)`	adata.obsm['spatial']: integer pixel coordinates from Space Ranger.	adata.obsm['spatial']: same array cast to float64.	Spatial distance computations (sq.gr.spatial_neighbors uses scipy.spatial.cKDTree) require floating-point input. Integer overflow can cause errors in distance calculations on high-resolution tissue images (pixel coordinates can exceed 32-bit integer range).

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`adata.var['mt'] = adata.var_names.str.startswith('MT-')`	adata.var.index: gene names as strings.	adata.var['mt']: boolean Series, True for mitochondrial genes (MT-CO1, MT-ND1, etc.).	Mitochondrial transcript fraction (pct_counts_mt) is a proxy for cell damage. Damaged cells lose cytoplasmic mRNA through ruptured membranes while retaining mitochondria (which have their own membranes), inflating mt fraction. High mt% spots are excluded as low-quality.
`adata.var['ribo'] = adata.var_names.str.startswith(('RPS','RPL'))`	adata.var.index: gene names.	adata.var['ribo']: boolean Series, True for ribosomal protein genes.	Ribosomal protein genes can account for 10–30% of total transcripts. Monitoring ribo% identifies spots with abnormal translational state (e.g., stress-induced ribosome biogenesis) and informs whether ribo genes should be regressed out in normalisation.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`sc.pp.calculate_qc_metrics(adata, qc_vars=['mt','ribo','hb'], ...)`	adata.X: raw integer count matrix. adata.var['mt'], adata.var['ribo'], adata.var['hb']: boolean masks.	adata.obs columns added: total_counts, n_genes_by_counts, log1p_total_counts, log1p_n_genes_by_counts, pct_counts_mt, pct_counts_ribo, pct_counts_hb.	Computes per-spot summary statistics used to define quality thresholds. total_counts = sum(X[spot, :]) is the library size. n_genes_by_counts = sum(X[spot, :] > 0) is the complexity. Both are used to identify empty spots, damaged cells, and doublets.
`sc.pp.filter_cells(adata, min_counts=500)`	adata.X with all spots. adata.obs['total_counts'].	Removes spots where total_counts < 500. adata.n_obs reduced accordingly.	Spots below 500 UMIs likely represent empty gel beads, slide artefacts, or tissue-less background. The count threshold is empirically set using violin plots of total_counts distribution; the threshold is placed at the inflection point of the low-count tail.
`sc.pp.filter_cells(adata, min_genes=200)`	adata.X after count filtering. adata.obs['n_genes_by_counts'].	Removes spots detecting fewer than 200 unique genes.	Gene complexity < 200 indicates either empty spots or heavily degraded RNA with near-zero biological information. 200 is a conservative lower bound; most tissue spots in a healthy section should detect 1,500–5,000 genes.
`adata = adata[adata.obs['pct_counts_mt'] < 20].copy()`	adata with pct_counts_mt computed for each spot.	Subsets adata to spots with < 20% mitochondrial reads. .copy() creates an independent object preventing view-based modification warnings.	20% mitochondrial threshold is tissue-type dependent (brain: 5%, heart: 30% due to high mitochondrial content). Spots exceeding this threshold are enriched for apoptotic or necrotic signal and will distort clustering towards a stress gene signature.
`sc.pp.filter_genes(adata, min_cells=10)`	adata.X after spot filtering.	Removes genes detected in fewer than 10 spots. adata.n_vars reduced accordingly.	Genes detected in very few spots provide minimal discriminative power for clustering while increasing computational burden. This filter removes noise-level, ultra-rare transcripts and reduces the feature space, accelerating downstream computation.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`adata.layers['counts'] = adata.X.copy()`	adata.X: raw integer sparse count matrix post-filtering.	adata.layers['counts']: a copy of the raw count matrix stored as a separate layer.	Preserving raw counts is mandatory for correct differential expression analysis. Methods such as DESeq2, edgeR, and scanpy's Wilcoxon test on raw counts assume the original UMI counts as input. Normalised values are inappropriate for these tests. AnnData.layers allows multiple simultaneous representations of the expression matrix.
`sc.pp.normalize_total(adata, target_sum=1e4, inplace=True)`	adata.X: raw integer count matrix.	adata.X: floating-point matrix where each spot i satisfies sum(X[i,:]) = 10,000. adata.uns['log1p']: dict tracking normalisation parameters.	Library-size normalisation corrects for technical variation in sequencing depth. Spot i receives normalisation factor f_i = 10,000 / sum(X[i,:]). Without this step, deeply sequenced spots would appear to express all genes at higher levels, creating false clustering by sequencing depth rather than biology.
`sc.pp.log1p(adata)`	adata.X: CP10K normalised floating-point matrix.	adata.X: element-wise log(x+1) transformed matrix. Values now in range [0, ~10].	Log transformation compresses the heavy right tail of count distributions (Negative Binomial / Poisson) towards approximate normality. The +1 pseudo-count prevents log(0). Without log-transformation, high-expression genes (e.g., ribosomal) dominate Euclidean distances and PCA components.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`sc.pp.highly_variable_genes(adata, flavor='seurat_v3', n_top_genes=3000, layer='counts')`	adata with raw counts in adata.layers['counts'].	adata.var columns added: highly_variable (bool), means, variances, variances_norm (standardised variance), highly_variable_rank.	Selecting 3,000 HVGs from ~20,000 genes reduces the feature space for PCA by 6.7x, removing genes that are constitutively expressed (low discriminative power) or stochastically expressed at noise level (high noise, low signal). Seurat v3 method fits a regularised negative binomial regression to model the mean-variance relationship, then selects genes with variance exceeding the model expectation.
`sc.pp.scale(adata, max_value=10)`	adata.X: log-normalised matrix.	adata.X: each gene column scaled to zero mean and unit variance. Values clipped to [-10, 10].	PCA computes principal components as linear combinations of features. Without scaling, genes with larger absolute expression values (e.g., ACTB, GAPDH) dominate the variance and pull PC1/PC2 towards themselves. Z-score scaling ensures each gene contributes equally. max_value=10 prevents outlier spots from distorting eigenvectors.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`sc.tl.pca(adata, n_comps=50, use_highly_variable=True, svd_solver='arpack', random_state=42)`	adata.X: scaled matrix. adata.var['highly_variable']: boolean mask.	adata.obsm['X_pca']: (n_spots × 50) PCA coordinate matrix. adata.varm['PCs']: (n_genes × 50) loading vectors. adata.uns['pca']['variance_ratio']: explained variance per component.	PCA finds orthogonal linear combinations of genes (principal components) that explain maximum variance. SVD decomposition: $X = U \Sigma V^T$. First 30–50 PCs capture the meaningful biological variance; later PCs are dominated by technical noise. This reduces dimensionality from ~3,000 HVGs to 50 PCs — a prerequisite for efficient k-NN graph construction.
`sc.pp.neighbors(adata, n_neighbors=15, n_pcs=30, random_state=42)`	adata.obsm['X_pca']: 50-dimensional PCA embedding.	adata.obsp['connectivities']: sparse (n×n) connectivity matrix. adata.obsp['distances']: sparse (n×n) distance matrix. adata.uns['neighbors']: parameter dictionary.	Constructs a k-NN graph where each spot is connected to its 15 nearest neighbours in PC space using Euclidean distance. The graph captures the local topology of the data manifold. Leiden clustering and UMAP both operate on this graph, not the raw expression matrix. n_pcs=30 is selected from the elbow plot.
`sc.tl.umap(adata, min_dist=0.3, spread=1.0, random_state=42)`	adata.obsp['connectivities']: k-NN graph.	adata.obsm['X_umap']: (n_spots × 2) 2D UMAP coordinates.	UMAP (McInnes et al., 2018) minimises the cross-entropy between the high-dimensional fuzzy topological representation and a low-dimensional embedding. min_dist controls how tightly points within clusters are packed; smaller values create tighter clusters. random_state=42 ensures reproducibility (UMAP uses stochastic gradient descent).
`sc.tl.leiden(adata, resolution=0.6, random_state=42, key_added='leiden')`	adata.obsp['connectivities']: k-NN connectivity graph.	adata.obs['leiden']: categorical Series, integer string cluster labels (0, 1, 2, ...).	Leiden (Traag et al., 2019) is a graph community detection algorithm that partitions nodes (spots) into communities that maximise modularity $Q = \Sigma[A_{ij} - k_i k_j/(2m)]$ where A is the adjacency matrix and k_i is the node degree. Resolution parameter γ scales the null model: higher $\gamma \rightarrow$ more communities. Leiden guarantees well-connected communities (fixes a known Louvain deficiency).
`sq.gr.spatial_neighbors(adata, coord_type='visium', n_neighs=6)`	adata.obsm['spatial']: physical pixel coordinates.	adata.obsp['spatial_connectivities']: sparse spatial adjacency matrix. adata.obsp['spatial_distances']: Euclidean pixel distances. adata.uns['spatial_neighbors']: metadata.	Constructs a spatial graph based on physical proximity (independent of transcriptomics). coord_type='visium' uses the known hexagonal lattice geometry of Visium spots (6 direct neighbours). This graph is used for neighbourhood enrichment and spatial autocorrelation analyses to detect spatially structured patterns.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`sq.gr.nhood_enrichment(adata, cluster_key='leiden', n_perms=1000, seed=42)`	adata.obsp['spatial_connectivities']: spatial graph. adata.obs['leiden']: cluster labels.	adata.uns['leiden_nhood_enrichment']['zscore']: (n_clusters × n_clusters) z-score matrix. adata.uns['leiden_nhood_enrichment']['count']: observed co-occurrence counts.	For each pair of clusters (A, B), computes the observed number of A-B spatial neighbour pairs and compares to the distribution from 1,000 random permutations of cluster labels. Z-score = (observed – mean_perm) / std_perm. Positive z-scores indicate spatial co-localisation (e.g., T-cells adjacent to tumour cells); negative z-scores indicate spatial exclusion.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`sc.tl.rank_genes_groups(adata, groupby='leiden', method='wilcoxon', layer='counts')`	adata with raw counts in adata.layers['counts']. adata.obs['leiden']: cluster labels.	adata.uns['rank_genes_groups']: nested dict with keys: names, scores, pvals, pvals_adj, logfoldchanges — each an (n_genes × n_clusters) recarray.	Wilcoxon rank-sum test: for each gene g and cluster c, tests H0: expression distribution in cluster c equals expression in all other clusters. Non-parametric — valid for overdispersed count data. Benjamini-Hochberg FDR correction applied to p-values. log2 fold-change computed as log2(mean_cluster / mean_rest + 1e-9). Raw counts avoid the Poisson noise amplification of normalised values.
`fig = plt.figure(figsize=(18, 14), dpi=300, facecolor='white')`	No existing figure.	fig: matplotlib Figure object, 18×14 inch canvas at 300 DPI (5400×4200 pixels).	300 DPI × 18 inches = 5,400 pixels wide — exceeding the 3,000-pixel minimum for print publication. facecolor='white' prevents transparent background artefacts in PDF/TIFF submissions.
`gs = gridspec.GridSpec(2, 3, ...)`	fig: empty Figure object.	gs: 2×3 GridSpec layout with explicit hspace, wspace, and margin padding.	GridSpec provides precise control over subplot dimensions and spacing, essential for publication figures where panels must align to journal specifications. hspace/wspace prevent panel label overlap; left/right/top/bottom margins are set to avoid clipping when the figure is exported.
`sc.pl.spatial(adata, color='leiden', ...)`	adata.obsm['spatial']: coordinates. adata.obs['leiden']: cluster labels. adata.uns['spatial']: tissue image.	Renders spots as coloured circles overlaid on the H&E; tissue image in ax_a.	Spatial cluster maps are the primary output of ST analysis — they directly answer 'where are these cell types?' The tissue image provides morphological context (tumour nests, vessel lumens, lymphoid follicles) that gives biological meaning to the cluster boundaries.
`fig.savefig(out_fig, dpi=300, bbox_inches='tight', facecolor='white')`	fig: completed Figure object with all 6 panels.	PDF and PNG files written to RESULTS_DIR.	bbox_inches='tight' crops the figure to its content boundary, preventing whitespace borders. dpi=300 in savefig overrides the screen rendering DPI to ensure the output file meets publication standards regardless of display settings. Saving as PDF provides vector graphics (infinitely scalable) for journal submission.

Line of Code	Input State	Output State	Scientific & Mathematical Rationale
`adata.write_h5ad(RESULTS_DIR / 'visium_analysed.h5ad')`	adata: fully analysed AnnData with all layers, embeddings, and results.	HDF5 file on disk containing the complete AnnData object.	HDF5 is the standard archival format for AnnData. It preserves all matrices (X, layers, obsm, obsp, varm, uns) in a single compressed file. This file is the primary deliverable for data deposition (GEO, Zenodo) and enables colleagues to reproduce all figures from a single file.

MODULE 6

Deriving Biological Conclusions

6.1 From Clustered Spatial Map to Scientific Abstract

The completion of the computational pipeline produces an annotated spatial cluster map: a tissue section in which each 55 µm spot is colour-coded by its transcriptomic community. This is the end of bioinformatics and the beginning of biology. The translation from visual cluster map to a publishable scientific claim requires a disciplined, multi-step interpretive framework.

6.1.1 Step 1: Morphological Co-registration

The first interpretive act is to overlay the cluster map on the H&E; tissue image and ask: *Do cluster boundaries correspond to recognisable histological structures?* A cluster that precisely coincides with lymphoid follicles suggests a lymphocyte population; a cluster coinciding with glandular epithelium suggests an epithelial identity. This morphological anchoring provides the first-pass biological hypothesis that motivates differential expression analysis.

6.1.2 Step 2: Differential Expression and Marker Gene Identification

For each Leiden cluster of interest, rank_genes_groups identifies the genes that are most specifically and significantly over-expressed in that cluster relative to all others. The key statistical outputs are:

- **Log2 Fold-Change (log2FC):** log2(mean expression in cluster / mean expression in rest). A log2FC > 1 indicates the gene is expressed at > 2x the level of other spots. log2FC < 0 indicates spatial depletion.
- **Wilcoxon p-value (adjusted):** Benjamini-Hochberg FDR-corrected p-value. Threshold: p_adj < 0.05. Note that with spatial data, spots within a cluster are not independent (spatial autocorrelation), so p-values may be anti-conservative.
- **Percentage expression (pts / pts_rest):** The fraction of spots in the cluster and outside the cluster expressing the gene. A good marker has pts > 0.5 and pts_rest < 0.2 — expressed in the majority of cluster spots but rare elsewhere.

6.1.3 Step 3: Cross-Reference Against Public Tissue Atlases

Marker gene lists are validated and interpreted through cross-reference with curated biological databases and tissue atlases:

- **Human Protein Atlas (proteinatlas.org):** Provides single-cell RNA-seq and spatial transcriptomics reference data for human tissues. Query each cluster's top marker genes to identify the matching cell type reference profile.
- **Allen Brain Cell Atlas:** For neural tissue, the Allen Brain Cell Atlas provides a spatial reference atlas with layer-resolved transcriptomic profiles from mouse and human brain. Cluster marker genes are matched against this atlas to assign cortical layer identity.
- **CellTypist:** An automated cell type annotation tool trained on curated scRNA-seq reference atlases. The command sc.tl.rank_genes_groups marker gene list is input to CellTypist for probabilistic cell type assignment.
- **Gene Ontology (GO) Enrichment:** Tools such as gprofiler2 or enrichR test whether a cluster's marker gene set is enriched for specific biological process terms (e.g., 'antigen presentation', 'oxidative phosphorylation', 'extracellular matrix organisation'), providing functional context beyond individual gene names.
- **GSEA / ssGSEA:** Gene Set Enrichment Analysis with curated gene sets from MSigDB (Hallmarks, C2 curated, C7 immunological signatures) identifies whether clusters show coherent enrichment of known functional programmes.

6.1.4 Step 4: Spatial Niche Definition and Neighbourhood Interpretation

With cell types annotated, the neighbourhood enrichment z-score matrix (Module 4, Panel E) reveals which cell types are spatially co-localised beyond random chance. Positive z-scores (e.g., z > 2) between a 'tumour epithelium' cluster and a 'cytotoxic T-cell' cluster identifies an immune-infiltrated tumour niche. Negative z-scores between the same tumour cluster and 'M2 macrophage' indicate macrophage spatial exclusion from the tumour core — a clinically relevant observation suggesting an immunosuppressive barrier mechanism.

These spatial relationships are quantified and reported as the core finding: not simply 'we found T-cells and macrophages' (achievable with bulk RNA-seq) but 'cytotoxic T-cells are enriched within 200 µm of PD-L1-expressing tumour cells, while M2 macrophages are confined to the tumour-stroma interface, forming a spatially distinct immunosuppressive perimeter' — a statement only possible with Spatial Transcriptomics.

6.2 Writing the Abstract and Discussion

A spatial transcriptomics paper abstract follows a canonical structure. The following template maps each sentence to the specific computational or biological result that generates it:

Abstract Component	Corresponding Analytical Output
Background sentence: Why is the spatial organisation of [tissue/disease] important?	Literature context. Motivates the need for ST over existing approaches.
'We profiled X tissue samples from Y patients using 10x Visium ST, generating N spots across M genes.'	adata.n_obs (N spots) and adata.n_vars (M genes) after QC filtering.
'Unsupervised Leiden clustering at resolution R identified K spatially distinct transcriptomic communities.'	adata.obs['leiden'].nunique() = K. Resolution parameter from sc.tl.leiden().
'Cluster X co-localised with [histological structure] and expressed canonical markers [gene1, gene2, gene3], consistent with [cell type] identity.'	sc.pl.spatial() overlay + sc.tl.rank_genes_groups() top markers + atlas cross-reference.
'Neighbourhood enrichment analysis revealed significant co-occurrence of [cell type A] and [cell type B] (z = +X.X, p < 0.001).'	sq.gr.nhood_enrichment() z-score matrix + sq.pl.nhood_enrichment() visualisation.
'Differential expression between spatially adjacent [niche] and [niche] clusters identified [N] genes...'	sc.tl.rank_genes_groups() with pval_cutoff=0.05 and log2fc_min=0.5.
'These findings suggest [mechanistic conclusion] with implications for [therapeutic/developmental context].'	Integration of spatial data, DE results, GO enrichment, and atlas annotations.

6.3 Validation and Reproducibility Standards

Biological conclusions from ST require independent validation to meet publication standards. The following complementary approaches are standard in high-impact ST papers:

- **RNAscope / smFISH Validation:** Single-molecule fluorescence in situ hybridisation (smFISH) provides sub-cellular resolution validation of ST-identified spatial expression patterns. Top 2–3 marker genes from each key cluster should be validated by smFISH on an independent tissue cohort, confirming that the computational spatial prediction corresponds to physical mRNA localisation.
- **Immunohistochemistry (IHC) / Immunofluorescence (IF):** Protein-level validation of top mRNA markers closes the Central Dogma loop. Co-staining with antibodies against the top marker genes and a morphological counterstain confirms spatial protein expression patterns predicted by the transcriptomic clusters.
- **Independent Cohort Replication:** ST datasets from independent patient cohorts, different tissue preparation protocols, or different spatial platforms (e.g., Xenium for sub-cellular resolution) that replicate the same spatial niches substantially strengthen the biological conclusion.

- **Public Data Deposition:** Raw FASTQ files must be deposited in GEO (accession prefix GSE) or ArrayExpress. Processed AnnData .h5ad files should be deposited in Zenodo or Figshare. Analysis code must be deposited in GitHub with a DOI-linked Zenodo archive. This triad of data, object, and code constitutes full reproducibility.
- **Computational Reproducibility:** Include a conda environment.yml or Docker image specification pinning exact software versions. ST results can vary with scanpy/squidpy minor versions due to algorithm updates; environment pinning ensures long-term reproducibility.

CONCLUSION Spatial Transcriptomics represents a paradigm shift in our ability to decode tissue architecture at the molecular level. By preserving the spatial address of every captured transcript, ST transforms the abstract cell type catalogue produced by scRNA-seq into a functional tissue map — revealing cellular neighbourhoods, morphogen gradients, and signalling axes that are mechanistically meaningful and therapeutically actionable. The pipeline presented in this playbook — from OCT cryosection to publication-grade figure — provides a complete, reproducible framework for the next generation of spatial biology discoveries.

— End of Spatial Transcriptomics Masterclass Playbook —